Практическая работа №3
Форматы данных

Предмет: Инструментальные средства разработки программного обеспечения
Выполнил: Богданов Роман Романович. Группа: ИС-41

Цель работы

Сравнить JSON, XML, YAML, MessagePack, CBOR и Protobuf по размеру, скорости и читаемости. Передать профиль пользователя по TCP и проверить чтение старых данных новой схемой Protobuf.

1. Ответы на вопросы

Текстовые форматы хранят данные в виде текста, поэтому их удобно читать и проверять вручную. Бинарные форматы используют специальные коды и компактное представление значений; для просмотра требуется программа-декодер. Размер и скорость зависят от структуры данных и реализации библиотеки.

Схемная эволюция - изменение структуры сообщения с сохранением совместимости версий. В Protobuf новые поля добавляют с новыми номерами, а старые номера не переиспользуют. Отсутствующее обычное строковое поле proto3 читается как пустая строка.

2. Подготовка данных и окружения

Создан каталог date_format со всеми скриптами из задания, исходной и сгенерированной схемой. Использован профиль из условия: id=42, Анна Смирнова, 22 года, студент, три курса, адрес Москва, индекс 101000. Запуск выполнен в Python 3.12.14. Версии библиотек закреплены в requirements.txt.

Таблица 1. Текстовые форматы

Формат	Размер, байт	Читаемость (1-5)	Комментарии
JSON	253	5	Нет
XML	283	3	Да: <!-- ... -->
YAML	182	5	Да: # ...

Здесь JSON записан с indent=2, как в step_json.py. В сводном сравнении используется JSON без отступов, как в run_all.py из задания. Поэтому размеры JSON в двух таблицах различаются; это результат форматирования, а не изменения данных.

3. Сравнение всех форматов

Формат	Байт	Читается человеком	Нужна схема
json	211	Да	Нет
xml	283	Да	Нет
yaml	182	Да	Нет
msgpack	157	Нет	Нет
cbor	159	Нет	Нет
protobuf	117	Нет	Да

Размеры относятся только к полезной нагрузке UTF-8/бинарного сообщения. Служебный заголовок нашего TCP-протокола занимает дополнительно 5 байт: 1 байт формата и 4 байта длины. Сетевые заголовки TCP/IP в таблицу не включены.

Измерение скорости

Формат	Сериализация, мкс	Десериализация, мкс
json	2.587	2.462
xml	25.211	13.699
yaml	203.724	338.312
msgpack	1.012	1.082
cbor	2.788	2.745
protobuf	2.942	3.911

Для каждой операции взята медиана пяти серий по 1000 повторений timeit. Файловая запись, печать и передача по сети в замер не входят. Для Protobuf учитывается преобразование Python-словаря в сообщение и обратно. Это результаты одного профиля на данном окружении, а не универсальный рейтинг форматов.

Проверка данных

Все шесть форматов успешно восстановили исходный профиль. Проверяются все поля, включая логический тип is_student, список courses и вложенный address. В XML типы id, age и is_student восстанавливаются явно, поскольку простой XML-текст сам не хранит типы Python.

4. Вывод скриптов JSON и Protobuf

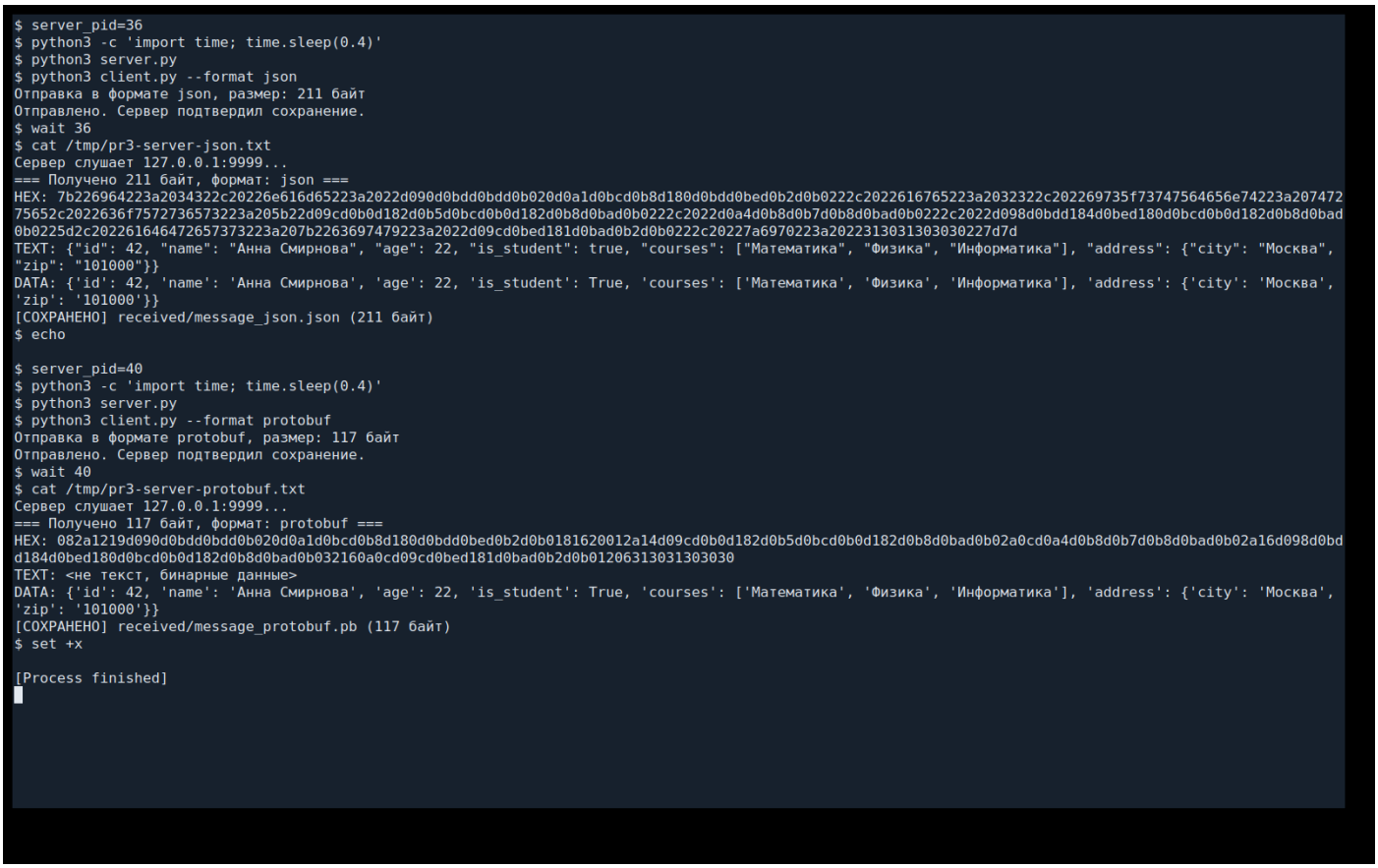
Ниже представлены настоящие снимки экрана терминала xterm при выполнении программ. Команды запущены повторно; захвачено изображение экрана, текст на снимках не дорисовывался.



5. Передача данных по TCP

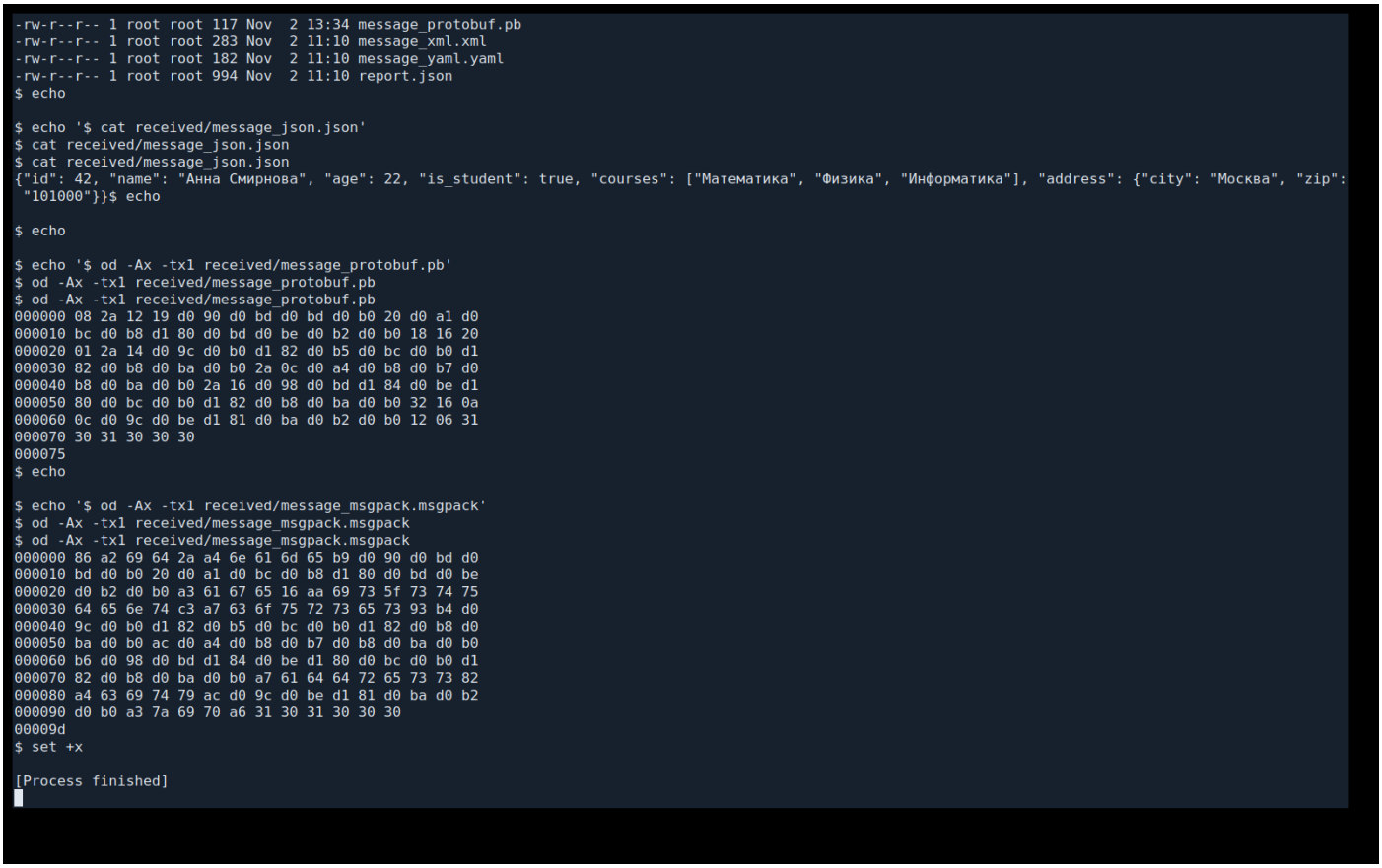
Клиент и сервер проверены отдельно для JSON, MessagePack, CBOR и Protobuf. Сервер принимает формат и длину, читает сообщение полностью, декодирует его, сохраняет в received/ и отправляет подтверждение OK. Для проверок использованы свободные локальные порты; обычный запуск использует 127.0.0.1:9999.

TCP передаёт поток байтов: один recv() может вернуть только часть заголовка. Поэтому функция recv_exact() дочитывает нужное число байт и сообщает об обрыве соединения. Дополнительно проверены разбиение заголовка на байты и преждевременное закрытие соединения.



6. Сохранённые файлы

В received/ сохранены сообщения всех шести форматов и сводный report.json. JSON открывается как UTF-8 текст, а Protobuf и MessagePack в этом примере не декодируются как обычный UTF-8 текст. Для бинарных файлов показано шестнадцатеричное представление.



7. Проверка схемной эволюции

Сначала скомпилирована старая схема legacy/profile.proto без email и сохранён message_protobuf.pb. Затем скомпилирована новая схема с полем string email = 7. Новый код прочитал старый файл: имя и остальные данные сохранились, значение email равно пустой строке. Дополнительно проверено совпадение байтов старого сообщения до и после остальных запусков.



8. Выводы

Protobuf оказался самым компактным: 117 байт вместо 211 байт у JSON без отступов. Он хранит номера полей вместо их полных имён и использует компактное кодирование значений. MessagePack занял 157 байт: имена ключей сохраняются, но значения и структура кодируются бинарно.

Вручную удобнее всего читать JSON с отступами; YAML также читается легко и позволяет писать комментарии. XML в этом примере более громоздкий из-за открывающих и закрывающих тегов. Самые быстрые операции в данном замере получились у MessagePack; результат зависит от библиотек и данных.

JSON сам не задаёт правила схемной эволюции, но совместимость можно организовать на уровне приложения: игнорировать неизвестные поля и задавать значения отсутствующих. Protobuf предоставляет такие правила через схему, постоянные номера полей и значения по умолчанию. Проверка показала, что добавление email с новым номером не мешает чтению старого сообщения.